

CENTRO UNIVERSITÁRIO SALESIANO - UNISALES

DISCIPLINA: MODELAGEM DE SISTEMAS E LEVANTAMENTO DE REQUISITOS

PROJETO SOMAR

Documentação Geral de Engenharia de Software

Compilado de Entregáveis Acadêmicos: Relatório Técnico de Desenvolvimento, Documento de Especificação de Requisitos (IEEE 830) e Modelagem de Diagramas UML (Casos de Uso, Classes, Atividades e Sequência).

PRODUTO: API REST BACKEND (SPRING BOOT)

Autor / Aluno: Mate (Desenvolvimento & Engenharia)

Parceiro de Par Programação: Antigravity AI

Data: 05 de Julho de 2026

SUMÁRIO

1. RELATÓRIO TÉCNICO	
1.1 Introdução	
1.2 Metodologia de Desenvolvimento	
1.3 Lições Aprendidas no Ciclo de Vida	
1.4 Conclusões e Trabalhos Futuros	
2. DOCUMENTO DE ESPECIFICAÇÃO DE REQUISITOS (IEEE 830)	
2.1 Introdução e Escopo do Produto	
2.2 Descrição Geral e Atores do Sistema	
2.3 Requisitos Funcionais (RF)	
2.4 Requisitos Não-Funcionais (RNF)	
3. MODELAGEM UML DO SISTEMA	
3.1 Diagrama de Casos de Uso (Atores e Ações)	
3.2 Diagrama de Classes (Estrutura e Relacionamentos)	
3.3 Diagrama de Atividades (Fluxo de Alocação)	
3.4 Diagrama de Sequência (Criação de Campanha)	

1. RELATÓRIO TÉCNICO — PROJETO SOMAR

Disciplina: Modelagem de Sistemas e Levantamento de Requisitos

Projeto: Somar (backend-somar)

Instituição: Centro Universitário Salesiano (UniSales)

Data: 05 de Julho de 2026

1. Introdução

O projeto **Somar** surge com o propósito de mitigar uma das principais dificuldades enfrentadas pelo terceiro setor: a captação de recursos e a conexão entre Organizações Não Governamentais (ONGs) e doadores em potencial. Diante de um cenário em que muitas ONGs realizam trabalhos sociais de extrema relevância, mas carecem de visibilidade e canais centralizados de arrecadação, a plataforma Somar atua como um elo digital facilitador.

A arquitetura de software implementada é baseada em uma API REST desenvolvida em **Java** com o framework **Spring Boot**, que serve como infraestrutura de retaguarda (backend). Essa API foi projetada para suportar operações de cadastro de usuários (ONGs, doadores e administradores), gerenciamento de informações institucionais das ONGs e a criação, controle e monitoramento de campanhas de arrecadação financeira. A estrutura de dados garante consistência e integridade ao vincular usuários a entidades organizacionais e gerenciar o ciclo de vida das campanhas por meio de regras de negócios bem delimitadas.

Neste relatório técnico, detalhamos a abordagem metodológica adotada no desenvolvimento do projeto, os desafios encontrados, as lições aprendidas e as considerações finais sobre os resultados alcançados com a implementação do sistema `backend-somar`.

2. Metodologia

O desenvolvimento da API Somar foi pautado pela adoção de práticas modernas de engenharia de software, integrando metodologias ágeis de gestão e uma arquitetura robusta e escalável baseada no padrão de camadas (Layered Architecture).

2.1 Processo de Desenvolvimento e Gestão Ágil

Para guiar o ciclo de vida do desenvolvimento do software, optou-se pela metodologia ágil **Scrum**, combinada com ferramentas visuais de gerenciamento de tarefas (como o Trello). O desenvolvimento foi estruturado em sprints de curta duração, permitindo feedbacks contínuos e ajustes incrementais rápidos. As principais etapas do desenvolvimento envolveram:

- Planejamento de Sprint (Sprint Planning):** Definição das histórias de usuário e divisão em tarefas técnicas (ex: modelagem de dados, implementação de repositórios, criação de endpoints e testes).
- Desenvolvimento Dirigido por Prioridade (MOSCOW):** Priorização das funcionalidades essenciais (*Must Have*), como cadastro de usuários, login simples e criação de campanhas, garantindo a entrega rápida de um Mínimo Produto Viável (MVP).

2.2 Arquitetura de Software e Padrões de Projeto

A API do backend foi estruturada utilizando a divisão clássica em camadas para promover baixo acoplamento e alta coesão:

- **Model (Entidades):** Representação do domínio da aplicação e mapeamento objeto-relacional (ORM) com JPA/Hibernate. As entidades principais são Usuario, Ong e Campanha.
- **Repository (Acesso a Dados):** Interfaces que estendem `JpaRepository`, encarregadas da persistência direta com o banco de dados relacional.
- **Service (Regras de Negócio):** Camada intermediária que abriga as validações lógicas do sistema. Por exemplo, impede a criação de uma campanha se a ONG associada não existir no sistema.
- **DTO (Data Transfer Objects):** Objetos de transferência de dados divididos em `Requests` e `Responses`, que evitam a exposição direta das entidades de banco de dados nos endpoints da API, melhorando a segurança e o encapsulamento.
- **Mapping (Mapeamento):** Mappers responsáveis pela conversão entre DTOs e Entidades, garantindo que o transporte de dados seja feito de forma organizada.
- **Controller (API REST):** Exposição dos recursos via protocolo HTTP, com rotas intuitivas para manipulação de recursos de usuários, ONGs e campanhas.

3. Lições Aprendidas

O processo de engenharia de requisitos e a implementação prática da API trouxeram ensinamentos fundamentais para a equipe. Os principais pontos de reflexão e aprendizado técnico incluem:

3.1 Gerenciamento de Relacionamentos no Banco de Dados

A modelagem dos relacionamentos do banco de dados representou um desafio inicial. O vínculo entre o usuário do tipo `ONG` e a tabela `ong` foi implementado como uma relação `@OneToOne` e `@ManyToOne`, respectivamente. Compreender as restrições de integridade referencial foi crucial: por exemplo, um registro de ONG não pode ser inserido sem que haja previamente um `Usuario` de tipo `ONG` cadastrado. Esse encadeamento lógico ensinou a importância de planejar a ordem de consumo dos endpoints do ponto de vista do frontend.

3.2 Utilização do Padrão DTO e Mapeamento Manual

Embora o uso de DTOs e Mappers aumente a quantidade inicial de código a ser escrito, a separação de responsabilidades mostrou-se extremamente benéfica. A API consegue retornar objetos enxutos no frontend (por exemplo, resumindo os dados da ONG ao listar campanhas, ocultando dados sensíveis como senhas e documentos corporativos). O mapeamento manual proporcionou total controle sobre os dados expostos, embora a equipe reconheça que, em projetos de maior escala, ferramentas como o MapStruct poderiam ser integradas para automatizar esse fluxo.

3.3 Segurança e Autenticação Basilar

A versão atual do backend utiliza autenticação Basic simples, enviando e-mail e senha por meio de requisição POST direta para login. No entanto, constatou-se que esse modelo possui restrições sérias de segurança para ambientes de produção. A equipe compreendeu a real necessidade de migrar para mecanismos de tokenização baseados em **JSON Web Token (JWT)** combinados com o **Spring Security**, o que facilitaria o controle de sessões e autorizações específicas por tipo de usuário ('UsuarioEnum.DOADOR' versus 'UsuarioEnum.ONG').

4. Conclusões

O projeto **Somar** alcançou os objetivos propostos ao fornecer uma solução de backend funcional, robusta e estruturada sob as melhores práticas do Spring Boot. A API REST implementada cobre as operações fundamentais necessárias para gerenciar a plataforma de doações, suportando fluxos coerentes de cadastro, login, vinculação institucional e listagem ativa de campanhas sociais.

O processo de modelagem de sistemas provou ser indispensável. Sem a correta identificação dos requisitos de domínio e a definição prévia das entidades ('Usuario', 'Ong', 'Campanha'), a implementação técnica estaria sujeita a retrabalhos decorrentes de falhas de arquitetura. O esforço dedicado à modelagem de dados e requisitos garantiu que o código resultante refletisse fielmente as necessidades dos atores da plataforma.

Como trabalhos futuros para a evolução do sistema, recomenda-se:

1. **Segurança Avançada:** Implementação do Spring Security com autenticação JWT.
2. **Integração de Gateway de Pagamento:** Simulação ou integração real com serviços de pagamento (como Pix ou cartões) para processamento imediato de doações no endpoint de campanhas.
3. **Paginação e Busca Avançada:** Implementação de paginação nos endpoints de listagem de ONGs e campanhas, bem como filtros de busca avançada por cidade, estado ou categoria de ajuda.
4. **Tratamento Global de Exceções:** Implementação de um manipulador de exceções ('@ControllerAdvice') para padronizar as mensagens de erro enviadas ao cliente frontend.

Com essas melhorias, a plataforma Somar estará plenamente capacitada para atuar em um ecossistema produtivo real, gerando impacto social positivo por meio da tecnologia.

2. DOCUMENTO DE ESPECIFICAÇÃO DE REQUISITOS

PROJETO SOMAR — PADRÃO IEEE 830

1. Introdução

1.1 Finalidade

Este documento tem como objetivo especificar formalmente os requisitos funcionais e não-funcionais da plataforma **Somar**. A plataforma visa prover um canal digital que conecta Organizações Não Governamentais (ONGs) a doadores individuais, facilitando a divulgação de campanhas beneficentes e a captação de auxílio financeiro ou material. A especificação segue as diretrizes da norma **IEEE 830-1998**, servindo como base técnica para desenvolvedores, testadores e demais partes interessadas.

1.2 Escopo do Produto

A API Somar (backend) viabiliza a criação de contas de usuário categorizadas por perfis de acesso ('ONG', 'DOADOR', 'ADMIN'), o gerenciamento cadastral de ONGs autorizadas e o lançamento de campanhas de arrecadação com metas financeiras específicas. O escopo do sistema abrange o ciclo de vida dessas campanhas (criação, ativação, encerramento ou cancelamento) e os fluxos de consulta pública de iniciativas sociais em andamento.

1.3 Definições, Acrônimos e Abreviações

- **API:** *Application Programming Interface* (Interface de Programação de Aplicação).
- **ONG:** Organização Não Governamental.
- **REST:** *Representational State Transfer*.
- **SRS:** *Software Requirements Specification* (Especificação de Requisitos de Software).
- **DTO:** *Data Transfer Object* (Objeto de Transferência de Dados).
- **IEEE:** *Institute of Electrical and Electronics Engineers*.
- **RF:** Requisito Funcional.
- **RNF:** Requisito Não-Funcional.

2. Descrição Geral

2.1 Perspectiva do Produto

A API Somar opera de maneira independente, respondendo a requisições via protocolo HTTP no formato JSON. A aplicação conecta-se a um banco de dados relacional para persistir os registros de usuários, dados cadastrais de ONGs e tabelas de campanhas. O frontend consome esses endpoints para exibir os dados de forma amigável ao usuário final.

2.2 Funções do Produto

As principais funções do sistema incluem:

- Gerenciamento de acessos (cadastro de usuários, validação de login).
- Gerenciamento de ONGs (registro completo de dados corporativos e relacionamento com usuário gestor).
- Gerenciamento de campanhas de doação (cadastro de campanhas por ONGs, controle de metas financeiras e alteração de status).

2.3 Características dos Usuários

O sistema possui três tipos principais de usuários:

1. **Doador (DOADOR):** Cidadãos interessados em apoiar projetos sociais. Podem buscar ONGs e campanhas ativas.
2. **Representante de ONG (ONG):** Responsável por cadastrar a organização e suas respectivas campanhas beneficentes, bem como editar ou finalizar as iniciativas lançadas.
3. **Administrador (ADMIN):** Perfil técnico com privilégios de auditoria e manutenção de dados do sistema, podendo remover registros inconsistentes.

2.4 Restrições Gerais

- A aplicação backend deve rodar obrigatoriamente no ambiente Java 17 ou superior.
- O tráfego de informações da API no ambiente local utiliza a porta `8080` (configurável).
- Os dados devem ser transmitidos exclusivamente no formato JSON.

3. Requisitos Específicos

3.1 Requisitos Funcionais (RF)

Esta seção lista em detalhes as funções que o sistema deve executar para atender às necessidades de negócios.

ID	Nome	Descrição	Atores	Prioridade
RF001	Cadastro de Usuários	O sistema deve permitir que novos usuários se cadastrem informando nome completo, e-mail, senha e o tipo de usuário (ONG, DOADOR ou ADMIN).	Visitante	Alta
RF002	Autenticação (Login)	O sistema deve autenticar o usuário validando as credenciais de e-mail e senha correspondentes, permitindo o acesso às áreas restritas do sistema.	Usuário	Alta
RF003	Cadastro de ONG	O sistema deve permitir que um usuário do tipo ONG registre os dados corporativos da organização, incluindo Nome, Descrição, Documento (CNPJ), Telefone, Cidade, Estado e o vínculo com o ID de seu usuário.	Usuário (ONG)	Alta
RF004	Listagem de ONGs	O sistema deve permitir a listagem pública de todas as ONGs cadastradas e ativas na plataforma para fins de consulta.	Todos	Média
RF005	Buscar ONG por ID	O sistema deve permitir a consulta detalhada de uma ONG específica a partir do fornecimento de seu identificador único.	Todos	Média
RF006	Exclusão de ONG	O sistema deve permitir que a organização ou um administrador delete o cadastro de uma ONG por meio de seu ID.	Usuário (ONG), ADMIN	Baixa
RF007	Criação de Campanha	O sistema deve permitir que uma ONG cadastrada crie uma campanha beneficente vinculada ao seu ID, informando Título, Descrição, Valor de Meta, Valor Atual (inicializado em zero), Data Limite de Finalização e Status Inicial (ATIVA).	Usuário (ONG)	Alta
RF008	Listagem de Campanhas	O sistema deve permitir a consulta e listagem de todas as campanhas criadas pelas ONGs, exibindo informações resumidas da organização promotora.	Todos	Alta
RF009	Alt. Status Campanha	O sistema deve permitir a atualização do status da campanha para os estados ATIVA, FINALIZADA ou CANCELADA conforme o andamento das metas.	Usuário (ONG), ADMIN	Média
RF010	Exclusão de Campanha	O sistema deve permitir a exclusão lógica ou física de uma campanha por meio do fornecimento do seu respectivo identificador.	Usuário (ONG), ADMIN	Baixa

3.2 Requisitos Não-Funcionais (RNF)

Os Requisitos Não-Funcionais definem as qualidades, restrições e características comportamentais da plataforma.

ID	Categoria	Descrição	Prioridade
RNF001	Segurança	As senhas dos usuários devem ser transmitidas de forma segura. Em versões futuras, o mecanismo de autenticação deve ser alterado de Basic Auth para JWT (JSON Web Tokens).	Alta
RNF002	Desempenho	Os endpoints de consulta devem responder a requisições padrão em um tempo inferior a 300 milissegundos sob condições normais de tráfego de rede local.	Média
RNF003	Portabilidade	A API REST do backend deve rodar de forma agnóstica a sistemas operacionais (Windows, macOS, Linux) graças ao uso da JVM.	Alta
RNF004	Usabilidade	A API deve seguir os princípios de design RESTful, retornando códigos de status HTTP apropriados (201 Created para inserções, 200 OK para consultas, 400 Bad Request, etc.).	Alta
RNF005	Integridade	O banco de dados deve impor integridade referencial nas tabelas, impedindo a criação de ONGs órfãs (sem usuário) e campanhas órfãs (sem ONG).	Alta
RNF006	Disponibilidade	A API deve ser projetada para apresentar alta disponibilidade no ambiente de produção, operando em modelo Stateless para facilitar o escalonamento.	Média

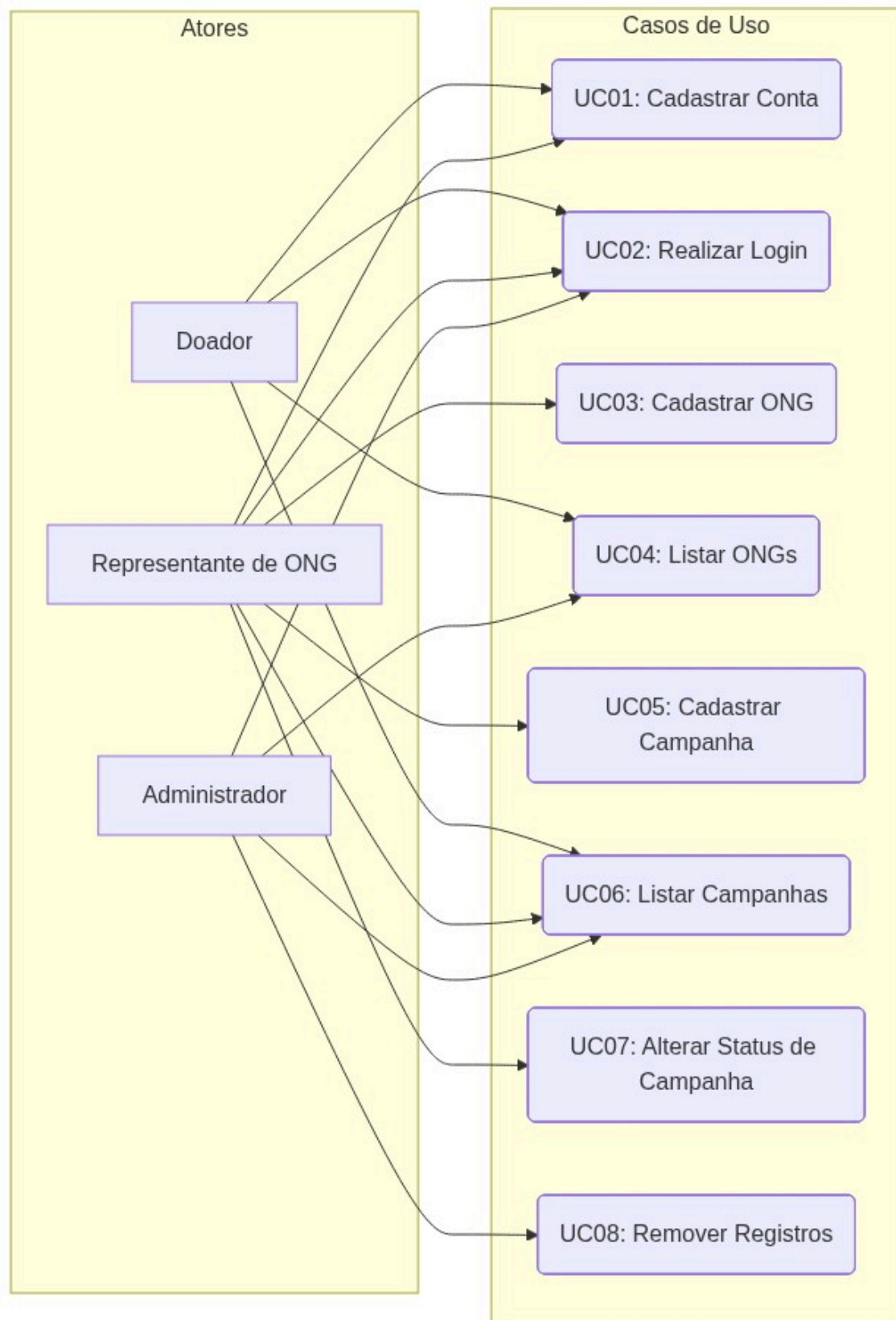
3. MODELAGEM UML DO SISTEMA — DIAGRAMAS

Este documento reúne a especificação visual dos diagramas UML do sistema **Somar**, modelados em formato nativo **Mermaid** para visualização direta no repositório.

1. Diagrama de Casos de Uso (Atores e Ações)

O diagrama abaixo descreve os atores do sistema (Doador, Representante de ONG, Administrador) e como eles interagem com as funcionalidades da plataforma.

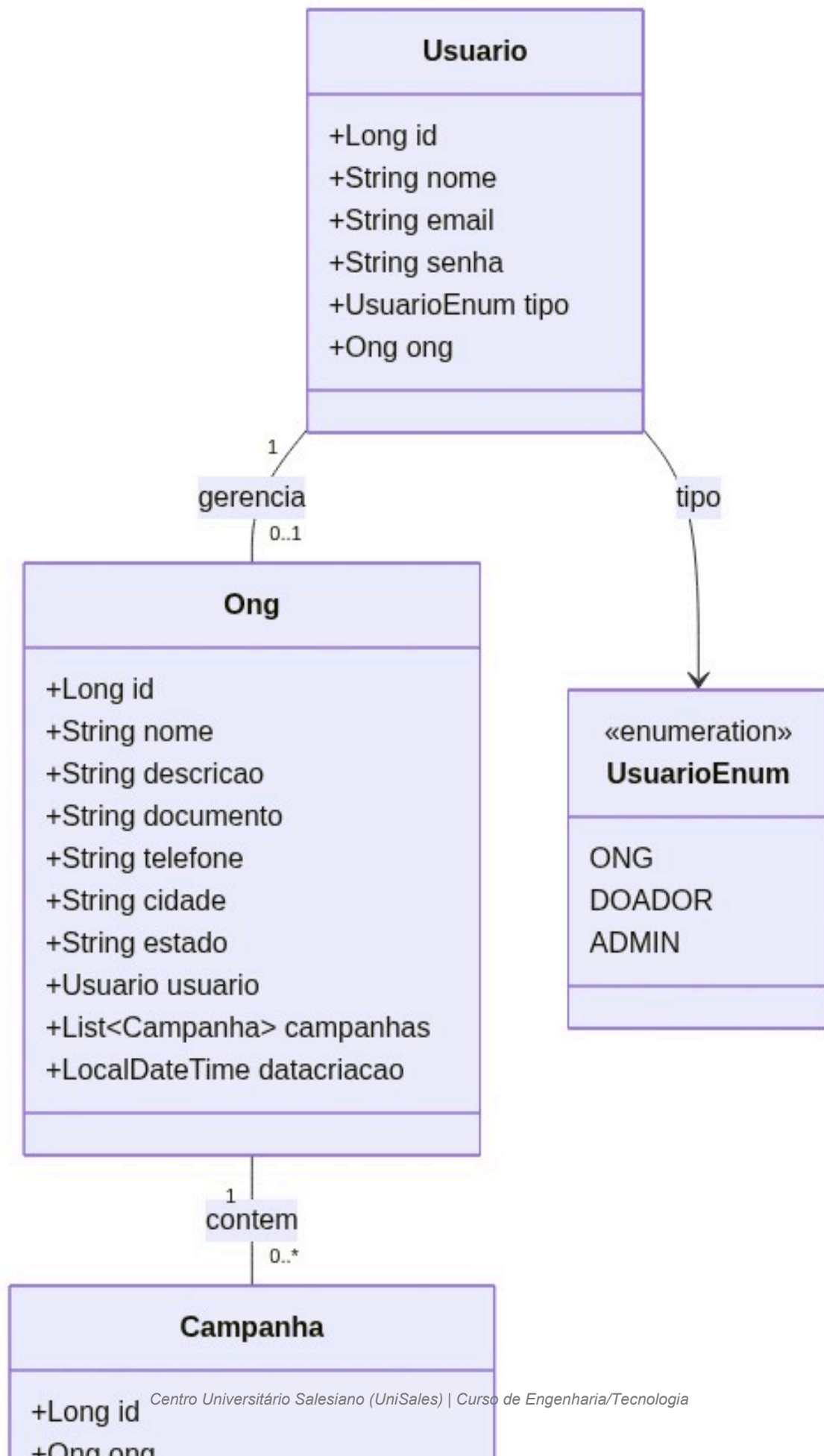
Relação Visual (Diagrama 1)



2. Diagrama de Classes (Atributos e Relações)

Este diagrama representa a estrutura de dados interna implementada no backend. Ele mapeia os atributos das classes de modelo Java (`Usuario`, `Ong`, `Campanha`) e suas relações de cardinalidade no banco de dados relacional.

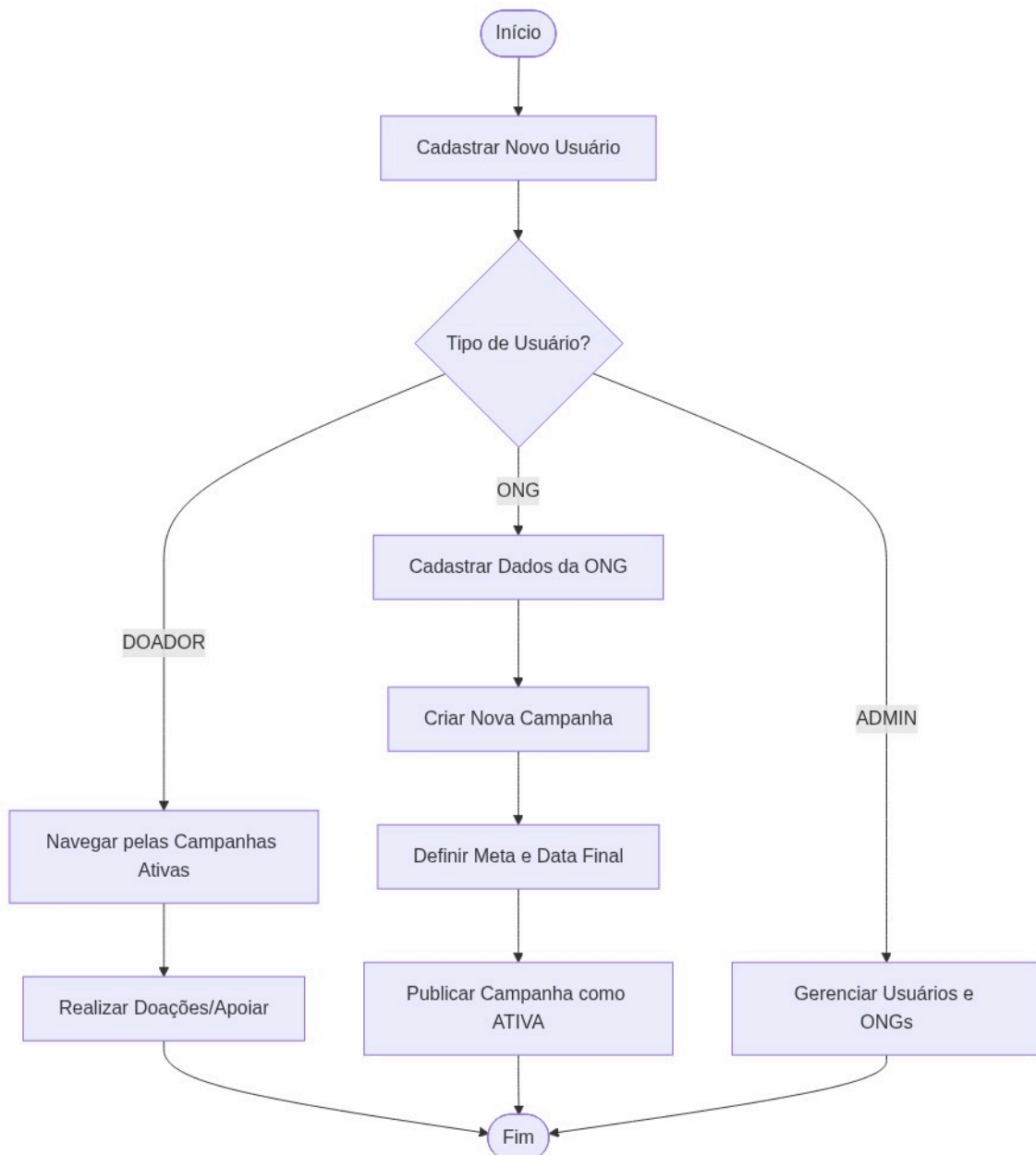
Relação Visual (Diagrama 2)



3. Diagrama de Atividades (Fluxo de Alocação/Criação)

Representa o fluxo lógico e os desvios condicionais que ocorrem desde o cadastro inicial de um usuário até a execução de suas atividades principais com base no perfil definido.

Relação Visual (Diagrama 3)



4. Diagrama de Sequência (Interação Usuário-Sistema)

O diagrama a seguir detalha o fluxo de mensagens e a interação síncrona entre o frontend (cliente) e as camadas internas do backend (Controller, Service, Repository, Banco de Dados) durante a criação de uma nova campanha beneficente por uma ONG.

Relação Visual (Diagrama 4)

